

ESP32 Audio Emotion Detection Project – Documentation

1. Introduction

This project investigates the feasibility of audio-based emotion recognition on a resource-constrained embedded platform (ESP32). The initial motivation was to explore whether emotionally significant acoustic events can be detected directly on-device, without reliance on cloud processing or external automated machine-learning platforms. At the current stage, the system implements and validates **anger detection**, while other emotional states (such as panic) remain open research directions.

The project was designed from the beginning as a fully **internal pipeline**. No end-to-end embedded ML frameworks (e.g., Edge Impulse or similar services) were used. Instead, all stages of the system—audio acquisition, signal preprocessing, feature extraction, dataset handling, model training, validation, and deployment considerations—were implemented and configured internally. TensorFlow is used only in the PC-based training stage as a neural network training framework; feature extraction and real-time constraints are defined independently and driven by embedded deployment requirements.

At this point in the project, multiple continuation paths are possible. The current implementation can be treated either as a sensing component within a larger IoT system or as an experimental platform for further research and performance analysis. The documentation below describes the work completed so far, independent of the project's eventual direction.

2. Development Process Overview

Development progressed iteratively, with design decisions continuously revised based on observed behavior in both offline and embedded-like conditions. Early stages focused on establishing a complete end-to-end pipeline that could realistically be deployed on the ESP32, rather than optimizing performance in an unconstrained desktop environment.

As the project evolved, emphasis shifted toward ensuring consistency between training-time assumptions and real-time embedded execution. This required revisiting feature extraction choices, data preprocessing steps, and validation methodology to ensure that the trained model operates on inputs representative of those encountered on the device.

3. Technical Challenges and Design Considerations

3.1 MFCC Consistency Between Training and Embedded Execution

A central challenge identified during development was a discrepancy between **MFCC features extracted in Python during training** and **MFCC features computed in C++ for embedded execution**. Models trained

on Python-extracted MFCCs initially showed acceptable performance during offline evaluation but exhibited degraded behavior when evaluated under embedded-like conditions.

This discrepancy was traced to implementation-level differences in MFCC computation, including windowing behavior, pre-emphasis filtering, mel filterbank construction, and numerical precision. Because MFCCs form the direct input to the neural network, these differences resulted in a mismatch between the feature distributions seen during training and those produced during deployment.

To address this issue, a research paper was identified that explicitly discusses MFCC alignment between Python and embedded C/C++ implementations and provides a corresponding GitHub repository. The Python-side MFCC extraction pipeline from this source was adopted and configured to match the embedded MFCC implementation. This alignment allows the training process to use features that more closely reflect those computed on the ESP32.

3.2 Offline Framing Versus Real-Time Audio Streaming

Another design consideration arises from the difference between **offline, frame-based audio processing** used during training and **continuous real-time audio streaming** on the embedded device. During training, complete audio signals are segmented into frames after acquisition. On the ESP32, audio is captured incrementally from a microphone and must be buffered and segmented in real time.

This difference introduces issues such as partial frames at buffer boundaries, temporal misalignment between consecutive frames, and sensitivity to hop size and buffering strategy. Frame length, hop size, and buffering behavior were selected based on stability under real-time streaming rather than convenience for offline processing.

3.3 Feature Selection Under Embedded Constraints

Feature extraction parameters were selected under explicit computational and memory constraints imposed by the ESP32. Sampling rate selection played a central role, as higher rates preserve more spectral detail but increase processing cost and memory usage. Lower sampling rates reduce complexity but limit available frequency information.

A deliberately low sampling rate was selected to balance real-time feasibility with retention of emotionally relevant acoustic information. This decision constrains the information available to the model and directly influences the types of emotions that can be distinguished reliably.

4. Dataset Construction and Preparation

4.1 Dataset Sources and Integration

The dataset used in this project was constructed by integrating multiple sources:

1. An existing dataset referenced in the selected research paper
2. Audio samples generated using text-to-speech (TTS), based on the project contributors' voices
3. Additional curated audio samples

These datasets differed in sampling rates, duration, recording conditions, and amplitude characteristics. A unified preprocessing pipeline was applied to standardize formats prior to feature extraction.

4.2 Data Cleaning and Label Handling

Data preparation included removing low-quality samples, trimming silence, and ensuring consistent labeling across merged datasets. This step was necessary to reduce the impact of noise and inconsistencies on the training and validation processes.

4.3 Synthetic Data and Dataset Bias

The inclusion of TTS-generated audio introduces potential dataset bias, as synthetic speech may contain regularities not present in natural recordings. In the current setup, TTS data is used as an augmentation source rather than a replacement for real recordings. Synthetic and real samples are combined during training to reduce over-reliance on artificial characteristics.

5. Model Training and Validation

Model training is performed in a PC environment using TensorFlow. The dataset is **randomly split into training and validation sets**, with the validation data strictly excluded from the training process. The validation set is used to monitor generalization behavior and evaluate the effect of changes to the dataset, feature extraction pipeline, and model configuration.

This validation approach ensures that reported performance reflects behavior on unseen data drawn from the same distribution, rather than memorization of the training samples.

6. Current Status and Open Directions

At the current stage, the project includes a complete embedded-oriented pipeline for audio-based anger detection, with aligned feature extraction between training and embedded execution and a defined validation methodology. The system serves both as a functional emotion-detection module and as an experimental platform.

Future continuation of the project may emphasize different aspects, such as system-level integration (e.g., treating the model output as a sensing signal within a larger IoT framework) or further research-oriented evaluation (e.g., comparative studies across datasets, feature configurations, or emotional classes). The work completed so far provides the technical foundation for either direction.